

Math-heuristic Wedding Planner

This assignment is the first math-heuristic assignment, relatively simple: Apply two given Julia/JuMP programs on a problem, a reduced complexity Wedding planning problem. The data are however the real data, from a wedding some years ago on the Danish island of Fyn. This larger set has 74 guests to be seated on 9 tables, each with 9 seats. This problem cannot be solved using the solvers directly (just try !). Hence there is a good argument for using a heuristic, which can find good but not guaranteed optimal solutions.

Model changes:

- Only consider the shared interests part of the objective. This problem is then a maximization problem
- Allow that some guests are un-seated, but penalize this heavily.

The used model is shown below

Problem 1

- Create a model which is suited for math-heuristic optimization and run the Hamming-Distance-Heuristic.

Sets

- The different guest: $g \in Guests = \{1, 2, \dots, 73, 74\}$
- The tables: $t \in Tables = \{1, \dots, 9\}$

Parameters

- $TableCap = 9$: The maximal number of guests sitting at each table
- $Couple_{g1,g2}$: Incidence matrix, where $Couple_{g1,g2} = 1$ if guests $g1$ and $g2$ are a couple
- $SharedInterests_{g1,g2}$: The number of shared interests of guests $g1$ and $g2$

Decision variables

- Is guest g seated at table t : $x_{g,t} \in \{0, 1\}$.
- Allow guests to **not** be seated: $slack_g \in \{0, 1\}$

Model

Objective:

- Maximize the number of shared interests.

$$\sum_{g1, g2, t | g1 < g2} SharedInterests_{g1, g2} \cdot y_{g1, g2, t} - 1000 \cdot \sum_g slack_g$$

Constraints:

- Now we allow that not all guest have to be seated, but un-seated guests are penalized in the objective:

$$slack_g + \sum_t x_{g,t} = 1 \quad \forall g$$

- Ensure that not too many guests are seated at each table:

$$\sum_g x_{g,t} \leq TableCap \quad \forall t$$

- Ensure that couples are seated at the same table:

$$x_{g1,t} = x_{g2,t} \quad \forall t, g1, g2 | Couple_{g1, g2} = 1$$

- Maximize the number of shared interests.

$$\sum_{g1, g2, t | g1 < g2} SharedInterests_{g1, g2} \cdot y_{g1, g2, t}$$

Below is the Julia/JuMP code:

```
start_time=time_ns()
#####
# WeddingPlanner Assignment, "Mathematical Programming Modelling" (42112)
# utilizing Hamming-distance heuristics optimization
#####
# Intro definitions
using JuMP
```

```

using HiGHS
#####

let
#####
# PARAMETERS
# Set: Guests, g,g1,g2
# Set: Tables, t
# DATA: SharedInterests[g1,g2]: 2-dim Guests*Guests: No of shared interests
# DATA: Couple[g1,g2]: 2-dim Guests*Guests: 1 if two guests are partners, zero-otherwise
# DATA: Age[g]: 1-dim, Guests: Age measured in years
# DATA: Male[g]: 1-dim, Guests: 1 if guest Male, 0 otherwise
# DATA: Female[g]: 1-dim, Guests: 1 if guest Female, 0 otherwise
# DATA: Know[g1,g2]: 2-dim Guests*Guests: 1 if two guests know each other (symmetric)
time_limit=120
include("WeddingData74.jl") # path to data
G=length(Guests)
T=length(Tables)
TableCap=ceil(G/T)
println("Runing Hamming-Distance-Heuristic on the WeddingPlanner with $(G) guests, $(T)
#####

#####
# Model
wp =Model(HiGHS.Optimizer)
set_silent(wp)
# 1 if guest g is sitting at table T
@variable(wp, x[g=1:G,t=1:T], Bin)

@variable(wp, slack[g=1:G], Bin)

# 1 if guest g1 and guest g2 are both sitting at table T, symmetric
@variable(wp, 0 <= y[g1=1:G,g2=1:G,t=1:T] <= ( g1 < g2 ? 1 : 0) )

# Minimize sum of different objectives
@objective(wp, Max,
    # maximize the negative total shared intersts
    sum( SharedInterests[g1,g2]*y[g1,g2,t]
        for t=1:T, g1=1:G, g2=1:G if g1<g2)

    # cost of not planning all guests
    - sum( G*slack[g] for g=1:G) )

# all guests has to sit at a table
@constraint(wp, [g=1:G],
    slack[g] + sum( x[g,t] for t=1:T) == 1)

```

```

# dont exceed the number of persons at a table
@constraint(wp, [t=1:T],
            sum( x[g,t] for g=1:G) <= TableCap)

# couples should sit at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; Couple[g1,g2]==1],
            x[g1,t] == x[g2,t])

# limit y, can only become 1 if g1 and g2 are at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; g1<g2],
            y[g1,g2,t] <= x[g1,t])

# limit y, can only become 1 if g1 and g2 are at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; g1<g2],
            y[g1,g2,t] <= x[g2,t])
*****

*****

function AddKConstraintAndOptimize(xVal,K)

    @constraint(wp, Kconstraint,
                sum( x[g,t] for g=1:G,t=1:T if xVal[g,t]==0) +
                sum( (1-x[g,t]) for g=1:G,t=1:T if xVal[g,t]==1)
                <= K)
    optimize!(wp)
    xVal=round.(Int,value.(x))
    slackVal=round.(Int,value.(slack))
    curObj=round(Int64,objective_value(wp))
    delete(wp, Kconstraint)
    unregister(wp, :Kconstraint)
    return (curObj,xVal,slackVal)
end

println("Starting Hamming Distance Heuristic")
xVal=zeros(Int8,G,T)
K=2
it=1
curObj=-1
oldObj=-1

while (time_ns()-start_time)/1.0e9<time_limit
    (curObj,xVal,slackVal)=AddKConstraintAndOptimize(xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(round(this_time,digits=1)) K: $(K) CurObj: $(curObj) Slack: $(slackVal)")
    it+=1
    oldObj=curObj
end

```

```

        if curObj<=oldObj # comparison for maximization problems
            K=K+1
        else
            K=2
        end
        oldObj=curObj
        it+=1
    end
    *****

    *****
println("Successfull end of $(PROGRAM_FILE)")
    *****
end

```

Using this algorithm (on my computer), I got the following results, when run for 120 seconds:

- It: 107 res: 290

Problem 2

- Create a model which is suited for math-heuristic optimization and run the Fix-and-Optimize-Heuristic.

Below is the Julia/JuMP code:

```

start_time=time_ns()
*****
# WeddingPlanner Assignment, "Mathematical Programming Modelling" (42112)
# optimized using FixAndOptimize
*****
# Intro definitions
using JuMP
using HiGHS
using Gurobi
using Printf
using Random
*****

let
    *****
    # PARAMETERS
    # Set: Guests, g,g1,g2
    # Set: Tables, t

```

```

# DATA: SharedInterests[g1,g2]: 2-dim Guests*Guests: No of shared interests
# DATA: Couple[g1,g2]: 2-dim Guests*Guests: 1 if two guests are partners, zero-otherwise
# DATA: Age[g]: 1-dim, Guests: Age measured in years
# DATA: Male[g]: 1-dim, Guests: 1 if guest Male, 0 otherwise
# DATA: Female[g]: 1-dim, Guests: 1 if guest Female, 0 otherwise
# DATA: Know[g1,g2]: 2-dim Guests*Guests: 1 if two guests know each other (symmetric)
time_limit=120
K=100
include("WeddingData74.jl") # path to data
G=length(Guests)
T=length(Tables)
TableCap=9 #ceil(G/T)
@printf "Runing WeddingPlanner with %d guests, %d tables with capacity %.0f\n" G T TableCap
*****

#*****
# Model
wp =Model(Gurobi.Optimizer)
set_silent(wp)
#set_optimizer_attributes(wp, "mip_abs_gap", 0.0)
set_attribute(wp, "time_limit", 60.0)

# 1 if guest g is sitting at table T
@variable(wp, x[g=1:G,t=1:T], Bin)

@variable(wp, slack[g=1:G], Bin)

# 1 if guest g1 and guest g2 are both sitting at table T, symmetric
@variable(wp, 0 <= y[g1=1:G,g2=1:G,t=1:T] <= ( g1 < g2 ? 1 : 0) )

# Minimize sum of different objectives
@objective(wp, Max,
    # maximize the negative total shared intersts
    sum( SharedInterests[g1,g2]*y[g1,g2,t]
        for t=1:T, g1=1:G, g2=1:G if g1<g2)

    # cost of not planning all guests
    - sum( G*slack[g] for g=1:G) )

# all guests has to sit at a table
@constraint(wp, [g=1:G],
    slack[g] + sum( x[g,t] for t=1:T) == 1)

# dont exceed the number of persons at a table
@constraint(wp, [t=1:T],
    sum( x[g,t] for g=1:G) <= TableCap)

```

```

# couples should sit at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; Couple[g1,g2]==1],
            x[g1,t] == x[g2,t])

# limit y, can only become 1 if g1 and g2 are at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; g1<g2],
            y[g1,g2,t] <= x[g1,t])

# limit y, can only become 1 if g1 and g2 are at the same table
@constraint(wp, [g1=1:G,g2=1:G,t=1:T; g1<g2],
            y[g1,g2,t] <= x[g2,t])
*****

*****

function FixAndOptimize(xVal,K)
    # first lock all values
    for g=1:G
        for t=1:T
            fix(x[g,t], xVal[g,t]; force = true)
        end
    end

    # then free K
    free_vars=0
    while free_vars<K
        g=rand(1:G)
        t=rand(1:T)
        if is_fixed(x[g,t])
            unfix(x[g,t])
            free_vars+=1
        end
    end

    optimize!(wp)
    xVal=round.(Int,value.(x))
    slackVal=round.(Int,value.(slack))
    cur_obj=round(Int64,objective_value(wp))
    return (cur_obj,xVal,slackVal)
end

println("Starting heuristic")
cur_xVal=zeros(Int8,G,T)

# set the fix and optimize neighborhood

```

```

#K=round(Int64,G*T/2)

it=1
cur_obj=-1
old_obj=-10000000000000000
this_time=(time_ns()-start_time)/1.0e9

while this_time<time_limit
    (cur_obj,cur_xVal,slackVal)=FixAndOptimize(cur_xVal,K)
    this_time=(time_ns()-start_time)/1.0e9
    println("It: $(it) Time: $(round(this_time,digits=1)) K: $(K) CurObj: $(cur_obj) OldObj: $(old_obj)")
    if cur_obj-old_obj < 0
        println("This should not happen")
        exit(0)
    end

    if cur_obj>old_obj+0.5 # comparison for maximization problems
        old_obj=cur_obj
    end

    it+=1
end
#####

#####
println("Successfull end of $(PROGRAM_FILE)")
#####
end

```

Running the Fix-and-Optimize heuristic, with different values of K, for 120 seconds gives:

- K: 100, It: 8009 res: 285
- K: 200, It: 4258 res: 307
- K: 300, It: 3359 res: 315
- K: 400, It: 2474 res: 329
- K: 500, It: 269 res: 331
- K: 600, It: 3 res: 328